

Using PyTorch to find patterns across alerts

What it measured, and what went on the dashboard instead

22 September 2026 · Prepared by Koustub

This is the idea we talked about: the rules detect single events, so use a neural model to read a *run* of alerts and judge it. I built it and measured it properly. The short version is that it loses, and this is the account of what it lost to and what I shipped instead.

A GRU over the alert sequence is beaten by plain logistic regression on all 8 held out networks. The neural model wins none of them, trailing by a mean of 0.049 average precision. On this evidence PyTorch is not justified for this problem, and I would rather report that than ship a neural network because it sounds better.

The data

The AIT Alert Data Set, published on Zenodo with the CSET 2024 paper under CC-BY-4.0. 2,600,263 Wazuh alerts from 8 simulated enterprise networks, each carrying a labelled multi-step intrusion, in native `alerts.json` form, which is the same record my own manager writes. I checked Kaggle and HuggingFace first; neither had real Wazuh alerts with ground truth attached.

What it cannot answer, stated up front. None of this lab's rules 100100 to 100113 have a parent signature in it. Rules 5501 and 5502 are the entire overlap. So this measures the method, not my ruleset. Only a campaign against the lab measures the lab.

Sampling

- An episode is a **300 second tumbling window**, labelled attack if it overlaps a ground truth phase: **8,915 episodes, 187 of them attack**, a base rate of 2.1%.
- **Whole networks held out, never the tail of a timeline.** A model scored on the last hour of a network it trained on may only have learned that network's noise floor.
- **8 folds, leave one network out**, three seeds per fold for the neural model. On a single split two of these finished 0.046 apart with forty attack episodes between them, well inside what a different pair of held out networks would move. Counting fold wins is the claim that survives.
- **Average precision, not f1 at a fixed cutoff.** At a 2% base rate the same logistic predictions score f1 0.049 at 0.5 and 0.367 at a threshold chosen on validation. Every threshold here was picked on validation and never on the test set.

What each model scored

The logistic regression, the average precision and the metrics are about sixty lines of numpy between them. The sequence model is PyTorch: an embedding over rule ids, a GRU, and a linear head.

MODEL	F1	AVERAGE PRECISION
Best single rule, the baseline	0.245 (sd 0.134)	0.161 (sd 0.092)
Logistic regression on shape and severity, deployed	0.192 (sd 0.107)	0.177 (sd 0.080)
GRU over the alert sequence	0.180 (sd 0.112)	0.199 (sd 0.081)
Logistic regression on rule counts and timing	0.292 (sd 0.121)	0.249 (sd 0.097)
Random, for reference	—	0.021, the base rate

All three find real signal: 0.249 against a 0.021 base rate is about 12 times better than chance at ranking. The sequence structure is simply not where the signal is.

What is deployed instead

The winning model's features are one column per dataset rule id. Pointed at this lab every one of those counts is zero, so I fitted a portable version on 11 features that name no signature at all: the shape of a burst and the severity inside it, which transfer because Wazuh levels are a property of the ruleset rather than of a network. It scores 0.177 against the winner's 0.249.

The size of that gap is itself the result. Portability costs 0.072 average precision and retains 71%. Most of the signal was in which rules fired, not in the shape of the burst, which is an argument for the rule based approach the lab already has rather than against it.

It runs inside the dashboard that already existed. No second dashboard, no new service, no new port, nothing installed on the manager: the scorer went into the status script the dashboard already ships to the manager on every poll. Alerts are bucketed into 300 second windows and each is scored, at a cost of 4 ms against a 90 ms poll budget.

A severity score, because the model answers the wrong question

The model says how unusual a window is. A queue needs how bad it is, and those are not the same. So each window also gets a 0 to 100 severity, a weighted composite with the model as one term among six: peak rule level, how much high level alerting there was rather than just the peak, how tight the busiest minute was, how many distinct signatures, and how many attack stages appeared. The total is multiplied by up to 30% when credential access and persistence both appear in the same window, because that pairing is a chain rather than two events.

A quiet window scores **16**, a brute force burst alone scores **54**, and the same burst with an account creation beside it scores **78**. The existing rules cannot separate those last two. They report both events and nothing that says they belong together.

Any window at 50 or above becomes a finding, expandable to the full working and exportable to PDF. Every constant is judgement rather than a fitted parameter, they are in one file, and the dashboard prints the formula beside the score.

The limit, and what would remove it

Fitted on eight public networks and never measured on this lab, whose own rules appear in no public dataset. Treat the score as triage ordering, not as a detection. A campaign is what would replace this sentence with a number.

That sentence is on the panel itself rather than buried in a document. The work that closes it is a long scenario campaign against the lab's own endpoints, producing labelled alerts that carry my rules, and that is the next thing I would do if this is worth continuing.

Eleven architecture diagrams covering the full system are in `docs/architecture/`, viewable as a single self-contained page. A longer seven page write up of the modelling, and the raw per-fold measurements, are available on request.